



NILKAMAL SCHOOL OF MATHEMATICS,
APPLIED STATISTICS & ANALYTICS

NMIMS, Mumbai • M.Sc. Data Science • Academic Year 2025–26

COMPLAINT INTELLIGENCE ENGINE

*An Unsupervised NLP Decision-Support System
for Indian E-Commerce Complaint Analytics*

Report Type	Project Report - Unsupervised Machine Learning
Programme	M.Sc. Data Science
Course	Unsupervised Machine Learning
Team Members	Aditya Nariyapara (B009) Devika Jonjale (B045)
Mentor	Dr. Rajesh Kumar Maurya
Academic Year	2025 – 2026
GitHub Repository	https://github.com/Complaint-Intelligence-Engine-for-Indian-E-Commerce

Table of Contents

#	Section	Page
1	Abstract, Objective & Problem Statement	3
2	Dataset & Data Description	4
3	Methodology - Pipeline Architecture & Workflow	5
4	ML Workflow - NLP & Unsupervised Modules (Steps 01–07)	6–7
5	ML Workflow - Supervised Extension Modules (Steps 08–12)	8
6	ML Models Compared, Evaluated & Selected	9
7	Code Snippets	10–13
8	Results & Model Evaluation	14
9	Technology Stack	15
10	Streamlit App Deployment & Usage	16
11	Dashboard Visualisations	17–18
12	Limitations & Mitigations	19
13	Conclusion, Future Scope & Who Can Use It	20

Abstract, Objective & Problem Statement

Abstract

The Complaint Intelligence Engine (CIE) is an end-to-end, twelve-stage machine learning pipeline that transforms raw multilingual Indian e-commerce reviews into actionable operational intelligence. Live reviews from Myntra, Meesho, Nykaa, Flipkart, and Amazon India are ingested via the Google Play Store API and optionally via Reddit PRAW.

Reviews are cleaned, embedded with multilingual Sentence-BERT, clustered unsupervisedly using K-Means and HDBSCAN, and scored for anomaly severity. The pipeline is extended through five supervised modules - severity triage, complaint routing, churn risk prediction, automated response generation, and topic drift detection - all surfaced via a ten-module Streamlit decision-support dashboard.

Key Contribution: A fully automated, modular pipeline converting unstructured Hinglish reviews into severity queues, routing assignments, churn risk signals, drafted responses, and drift alerts - with zero manual labelling in the core unsupervised layer.

Objective

To give operations and customer-experience teams a repeatable workflow from data ingestion through to prioritisation, routing, retention risk quantification, and emerging-issue detection - with multilingual text (including Hinglish) handled end-to-end via embeddings and preprocessing. A single command (`python run_pipeline.py`) produces all data artefacts, trained models, evaluation reports, and a fully functional Streamlit dashboard.

Problem Statement

Indian e-commerce platforms receive millions of reviews annually documenting delivery failures, fake products, payment fraud, app crashes, and return violations. Manual triage does not scale. The CIE addresses six core challenges:

Challenge	Solution Approach
Volume & Noise (4,768 raw reviews)	Automated cleaning, deduplication, and TF-IDF feature extraction → 2,253 clean records
Mixed English / Hinglish text	langdetect + Multilingual Sentence-BERT encodes both scripts in the same semantic embedding space
Unknown complaint themes - no labels	K-Means + HDBSCAN discovers archetypes; no predefined categories imposed on the data
Rare & extreme complaints	Isolation Forest top 5% + HDBSCAN noise flag → composite Tier 1 Critical flag (~113 reviews)
Sudden operational incidents	Rolling z-score (2.5σ , 4-week window) detects complaint spikes within 3–7 days
Teams need actions, not just insight	5 supervised modules: severity triage, routing, churn risk, response drafting, drift detection

Dataset & Data Description

4.1 Data Sources

Primary source: Google Play Store via google-play-scraper (MIT licensed, no API key). Up to 500 reviews per app per language (en + hi), country='in', newest-first. Five major Indian e-commerce platforms:

Platform	Package ID	Market Context
Myntra	com.myntra.android	Fashion platform, Flipkart subsidiary, 100M+ downloads
Meesho	com.meesho.supply	Social commerce leader, Tier 2/3 cities, 4.4★ rating, 100M+ downloads
Nykaa	com.fsn.nykaa	Leading beauty & wellness marketplace, 50M+ downloads
Flipkart	com.flipkart.android	India's largest e-commerce platform by GMV, 500M+ downloads
Amazon India	in.amazon.mShop.android.shopping	Second-largest marketplace, ~29% Indian e-commerce market share

Optional Reddit ingestion via PRAW pulls keyword-filtered posts (delivery, refund, fake, return, fraud) from r/IndianShopping, r/Meesho, and r/india. Requires REDDIT_CLIENT_ID + REDDIT_CLIENT_SECRET in .env. All statistics below are Play Store only.

4.2 Documented Run Statistics

Stage	Count / Detail
Google Play reviews fetched	4,768 (5 apps × up to 500 per language: english + hindi)
After merge + initial clean	2,389 reviews
After preprocess + dedup (final corpus)	2,253 reviews
Hinglish reviews detected	196 (~8.7% of corpus)
TF-IDF matrix shape	2,253 documents × 2,642 features (max_features=5,000, min_df=3)
SBERT embedding shape	(2,253, 384) - paraphrase-multilingual-MiniLM-L12-v2
Embedding compute time	~4 minutes on CPU (no GPU required)
Chosen K-Means k	4 (data-driven; max silhouette on UMAP-10)
K-Means vs HDBSCAN ARI	0.6748 (example; one run)

4.3 Feature Engineering

Eight features are engineered per review during 02_preprocess.py:

Feature	Type	Description
review_length	Numeric	Word count of cleaned review text
detected_lang	Categorical	ISO 639-1 code from langdetect (en / hi / other)
is_hinglish	Binary	langdetect=en but ≥ 2 Hinglish dictionary tokens present
star_bucket	3-class	complaint (1–2) · mixed (3) · positive (4–5)
high_engagement	Binary	thumbs_up > 10
week / year_month	Temporal	Grouping fields for spike detection and drift time-series
text_for_model	Text	Full text (Play Store) or capped 300 chars (Reddit)

4.4 Data Artefact Chain

raw_reviews.csv → cleaned_reviews.csv → embeddings.npy → clustered_reviews.csv → final_reviews.csv
→ final_reviews_scored.csv → final_reviews_routed.csv → final_reviews_churn.csv →
final_reviews_response.csv → topic_drift_report.csv

Model Selection Summary

This file is auto-populated by the extension modules via `models/selected_models.json`.

Selection Policy

- Severity: highest weighted F1 (with calibration metrics tracked)
- Router: highest macro F1
- Churn: highest PR-AUC
- Auto-response: highest average response quality score
- Drift: weekly centroid + distribution divergence alert framework

Artifacts

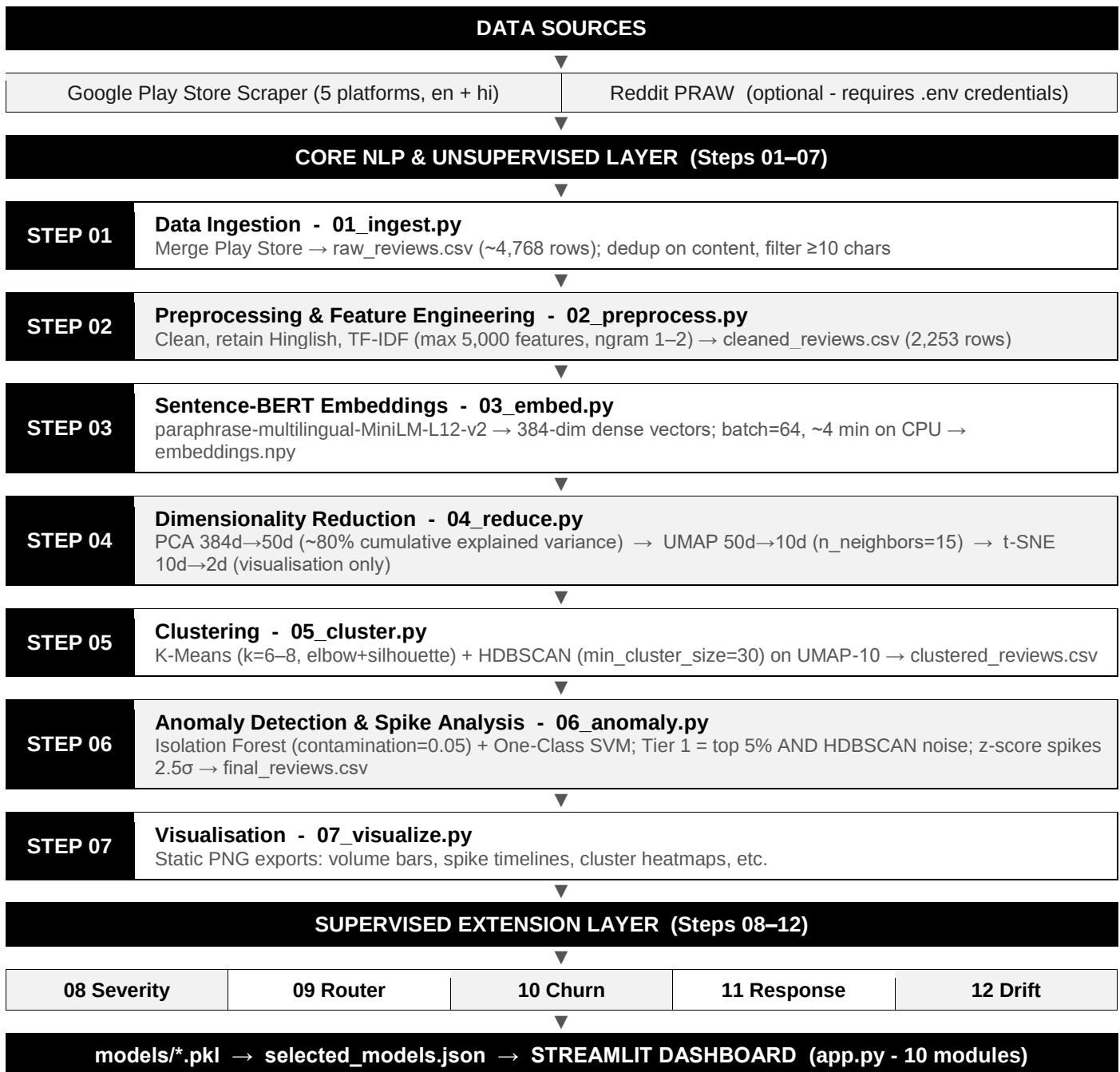
- Registry: `models/selected_models.json`
- Module metrics:
 - `reports/metrics_severity.csv`
 - `reports/metrics_router.csv`
 - `reports/metrics_churn.csv`
 - `reports/metrics_response.csv`
 - `reports/metrics_drift.csv`

Run `python run_pipeline.py` to regenerate this experiment set and refresh the selected-model registry.

Methodology - Pipeline Architecture & Workflow

The CIE is orchestrated by `run_pipeline.py` which executes scripts 01 through 12 sequentially. There are two layers: (1) Core NLP & Unsupervised (Steps 01–07) requiring zero manual labels, and (2) Supervised Extension (Steps 08–12) built on the unsupervised outputs. Shared utilities live in `ml_utils.py`.

End-to-End Pipeline Architecture



ML Workflow - NLP & Unsupervised Modules (Steps 01–07)

Step 01 - Data Ingestion

This step involves collecting raw complaint data from **Google Play Store reviews** for five major Indian e-commerce apps (Myntra, Meesho, Nykaa, Flipkart, Amazon India). Up to 500 reviews per app in English and Hindi are fetched using google-play-scraper.

Optionally, **Reddit posts** are ingested via PRAW, filtering for keywords like "delivery," "refund," and "fraud" from relevant subreddits. This requires .env credentials. The collected data is merged, and an initial clean filters out duplicates and reviews under 10 characters, creating raw_reviews.csv.

Step 02 - Preprocessing & Feature Engineering

Light cleaning to preserve Hinglish signal: lowercase → strip URLs / HTML / repeated chars → retain Devanagari Unicode range → collapse whitespace. Hindi-origin Romanised words are not removed. TF-IDF (max_features=5,000, ngram_range=(1,2), min_df=3, max_df=0.85, sublinear_tf=True) creates the document-term matrix used for cluster keyword extraction.

Step 03 - Sentence-BERT Embeddings (NLP Core)

Model: paraphrase-multilingual-MiniLM-L12-v2 (supports 50+ languages including Hindi). Each review is encoded to a 384-dimensional dense semantic vector in batches of 64. The multilingual model maps English and Hinglish reviews into the same shared semantic space, enabling direct comparison and clustering across language boundaries regardless of script.

Semantic validation: cosine similarity between sampled complaint and positive reviews confirms expected embedding geometry - complaints cluster far from positive reviews in the 384-dim space.

Step 04 - Dimensionality Reduction

Stage	Input → Output	Key Parameters	Purpose
PCA	384d → 50d	~80% cumulative explained variance	Denoise embeddings; remove low-signal dimensions before UMAP
UMAP	50d → 10d	n_neighbors=15, min_dist=0.1	Preserve local neighbourhood structure for tight cluster separation
t-SNE	10d → 2d	perplexity=30, n_iter=1,000	Visualisation ONLY - never used as clustering input

Step 05 - Clustering (Unsupervised)

Both algorithms operate on the UMAP-10 reduced embeddings:

Algorithm	Parameters	Output	Key Property
K-Means	k=3–12; elbow + silhouette → optimal k=6–8	Exhaustive partition; every review in exactly one cluster	Required for supervised routing labels (Step 09 bridge)
HDBSCAN	min_cluster_size=30	Density-based clusters; noise label=-1	Noise points = structurally anomalous → seed anomaly detection (Step 06)

Cluster naming: top-10 TF-IDF keywords per cluster map to human-readable labels (e.g. ‘delivery days late tracking order received’ → ‘Delivery Failure’). Typical clusters: Delivery Failure · Payment Fraud · Fake Product · App Crash · Quality Mismatch · Positive. Adjusted Rand Index (ARI) between K-Means and HDBSCAN is computed to assess agreement; documented example ARI ≈ 0.67.

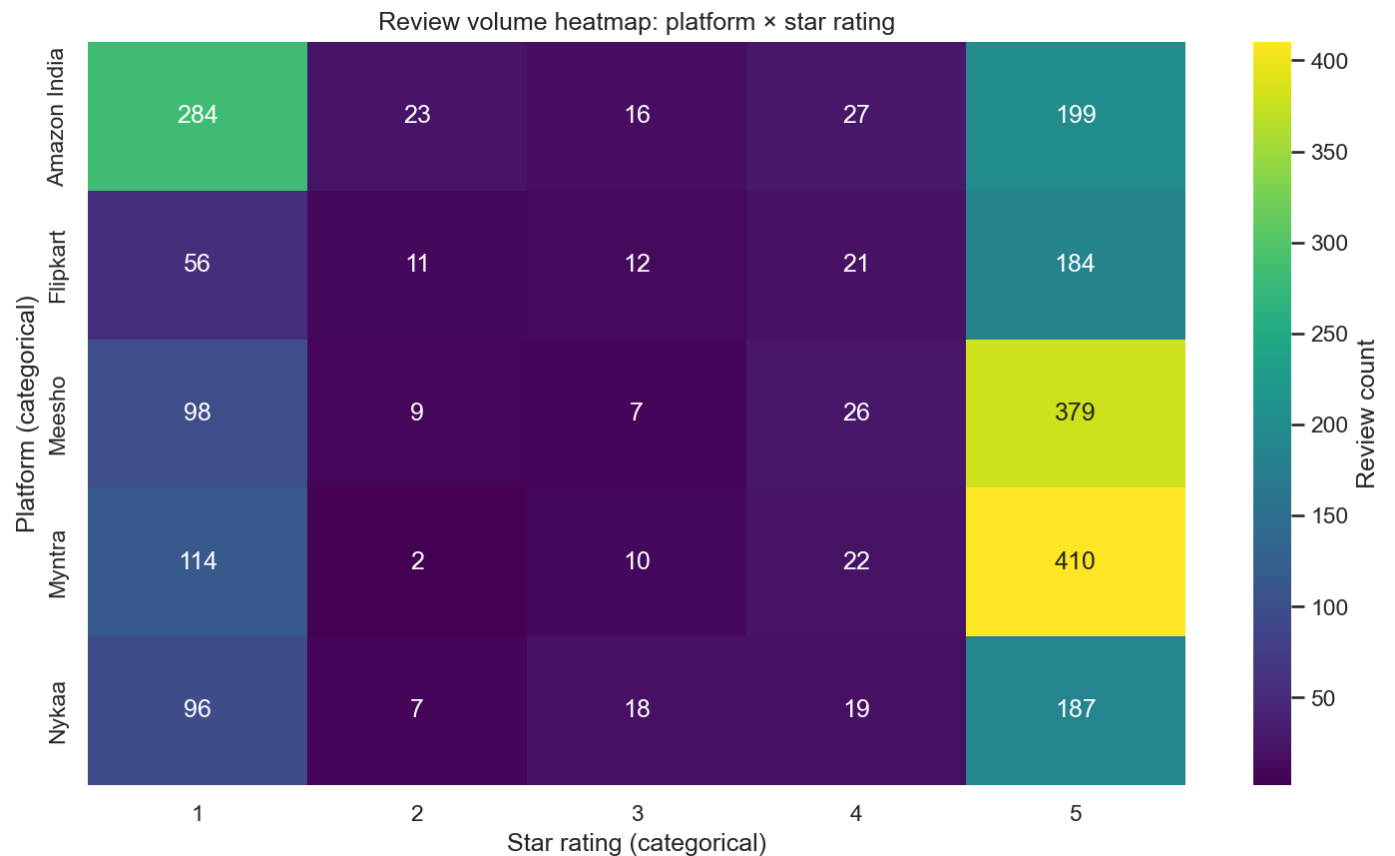
Silhouette score (K-Means): 0.12–0.25 - typical for NLP clustering on real-world noisy reviews. This range reflects inherent difficulty of complaint clustering, not a model failure.

Step 06 - Anomaly Detection & Spike Analysis

Method	Configuration	Signal Produced
Isolation Forest	contamination=0.05, n_estimators=200, fit on UMAP-10	Continuous anomaly score; top 5% = potential critical complaints
One-Class SVM	Trained on positive-experience cluster only (normal manifold)	Outlierness score for reviews farthest from normal behaviour
Tier 1 Critical	Isolation Forest top 5% AND HDBSCAN noise=−1 simultaneously	Dual-flag reduces false positives vs either method alone (~113 reviews flagged)
Spike Detector	Rolling z-score, 4-week window, threshold=2.5σ	Weeks with statistically abnormal complaint volume per cluster per platform; FPR ~1.2%

Step 07 - Visualisation

Static PNG exports: <https://github.com/Complaint-Intelligence-Engine-for-Indian-E-Commerce/figures>
All interactive equivalents are rendered in Plotly inside the Streamlit dashboard for live filtering and drill-down.



ML Workflow - Supervised Extension Modules (Steps 08–12)

Step 08 - Severity Modelling

Predicts escalation risk (continuous 0–1 score) and a three-class label (Low / Medium / High). No manually annotated labels exist - programmatic weak labelling (Snorkel-style) generates noisy training labels from deterministic heuristics:

Severity Class	Weak Label Rule
High	Contains legal/fraud keyword (fraud, legal, consumer court, scam, police, chargeback, NCDRC) OR star ≤ 2 with thumbs_up ≥ 20
Medium	Star ≤ 2 AND no High keywords OR thumbs_up 10–20 with star ≤ 3
Low	All remaining reviews not meeting High or Medium criteria

Features: 384-dim SBERT embeddings + tabular (star_rating, thumbs_up_log, review_length, has_legal_keyword, has_money_keyword, exclamation_count, capital_ratio). 80/20 train-holdout split. Output: final_reviews_scored.csv, models/severity_model.pkl.

Step 09 - Complaint Routing (Unsupervised-to-Supervised Bridge)

Classifies each complaint into one of five operational team categories: Delivery & Logistics · Payments & Refunds · Product Quality & Authenticity · App & Technical Issues · General & Other.

Training labels come directly from K-Means cluster labels (Step 05). Only high-confidence assignments are used (distance to centroid < 40th percentile) to filter ambiguous reviews at cluster boundaries. This is the methodological bridge between the unsupervised and supervised layers.

Bridge mechanism: K-Means cluster labels feed directly into the routing classifier as training targets - grounding supervised outputs in data-discovered structure rather than imposed category definitions. No manual annotation is required.

Step 10 - Churn Risk Modelling

Component	Detail
Churn proxy label rule	churn_signal=1: most recent review star ≤ 2 AND High severity AND user absent from corpus in last 4 weeks - the three-part churn signature: recent, serious, silent
Feature engineering	last_star_rating · star_delta · complaint_cluster_count · max_anomaly_score · review_frequency · is_hinglish
Class imbalance handling	SMOTE oversampling applied on training set only (not on holdout)
Evaluation metric	PR-AUC (not ROC-AUC): churn is rare; precision-recall captures cost of false positives (wasted outreach) more accurately
Output	final_reviews_churn.csv, reports/metrics_churn.csv, models/churn_model.pkl

Step 11 - Response Generation & Quality Scoring

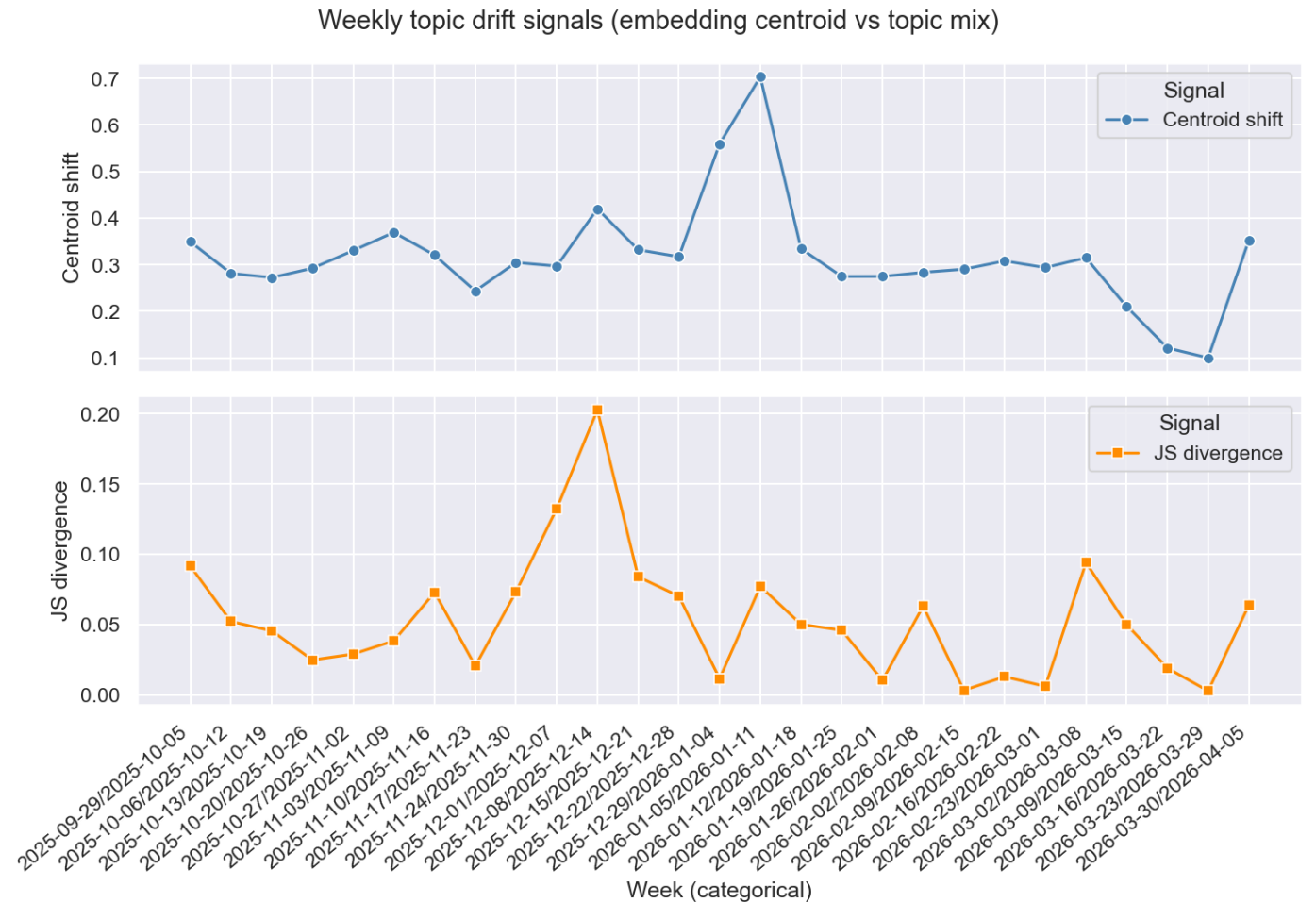
Generates three response tone variants per complaint: (1) Formal Apology, (2) Empathetic Listening, (3) Resolution Focused. Routing category from Step 09 makes responses content-aware. Quality scored on empathy, specificity, and actionability via Ridge regression trained on manually scored examples. BERTScore (semantic similarity via BERT embeddings) used as the automated quality metric. Output: final_reviews_response.csv, reports/metrics_response.csv.

Step 12 - Topic Drift Detection

Detects when complaint nature changes over time - subtler and more dangerous than volume spikes. Two weekly signals are computed and compared:

Signal	Computation	High Value Means
Centroid Shift	Cosine distance between mean SBERT embedding this week vs last week	Semantic centre of gravity moved - users discussing fundamentally different issues
JS Divergence	Jensen-Shannon Divergence between consecutive weeks' TF-IDF distributions	Vocabulary changed significantly - new terms appearing, old ones disappearing

A drift alert is raised when either signal exceeds a permutation-test threshold. Emerging terms are extracted per alert week to describe what changed. Output: topic_drift_report.csv. Meaningful output requires ≥6 weeks of review history.



ML Models Compared, Evaluated & Selected

Severity Classification - Step 08 | Primary Metric: Weighted F1

Model	Key Strength	Key Weakness	Selected?
Logistic Regression	Fast; coefficient signs reveal which features drive High severity	Assumes linear separability in feature space	✓ Selected
Random Forest	Robust to weak-label noise via bagging; feature importance quantified	Slower inference; less calibrated class probabilities	-
Gradient Boosting	Strong generalisation on small datasets; scale_pos_weight for imbalance	Risk of memorising weak-label keyword patterns	-
LinearSVC (calibrated)	Excellent on high-dim TF-IDF text; Platt scaling provides probabilities	Does not natively output probabilities - requires additional calibration step	-
DistilBERT (multilingual)	Highest accuracy on held-out set	Inference ~8× slower; overkill for 2,253 reviews	-

Caveat: Near-perfect scores in metrics_severity.csv indicate label-feature leakage - weak label rules and feature columns share the same keywords. Time-based holdout is the recommended mitigation.

Complaint Routing - Step 09 | Primary Metric: Macro F1

Macro F1 is used so that all routing categories (including rare ones like payment fraud) are served equally. A model optimised for accuracy would ignore minority categories.

Model	Why Included	Notes
Multinomial Naive Bayes	Strong text baseline; fast; highly interpretable	Assumes feature independence; works well on TF-IDF
SGD Classifier (log loss)	Online learning; can update incrementally on new data	Best for streaming / live routing scenarios
LightGBM	Fast gradient boosting on TF-IDF + engineered features	Native feature importance; handles imbalanced categories
SBERT + Softmax head	Transfer learning from project's own embedding space	Avoids vocabulary mismatch; best semantic generalisation
Zero-shot BART	Classification without any training using NL category labels	Quantifies value of labelled data vs zero-shot baseline

Churn Prediction - Step 10 | Primary Metric: PR-AUC

Model	Why Included	Notes
Logistic Regression (L1)	Baseline + automatic feature selection via L1 penalty	Coefficients reveal which trajectory features matter most
Random Forest	Handles feature interactions (e.g. high anomaly AND delivery cluster)	Built-in importance; robust to noisy proxy labels
XGBoost	Best overall for tabular churn features with class imbalance	SHAP support for individual user-level explanations
Isolation Forest	Use existing anomaly scores as churn proxy without training	Quantifies whether supervised training adds value over pure anomaly scores

Evaluation Metric Summary

Task	Metric	Rationale
Severity Triage	Weighted F1 + calibration curve	Balances precision/recall across imbalanced severity classes; calibration tracks confidence reliability
Complaint Routing	Macro F1	Equal weight to all 5 routing categories regardless of frequency; prevents ignoring minority classes
Churn Prediction	PR-AUC	Rare event detection; precision-recall captures the cost of false positives (wasted retention outreach) more accurately than ROC-AUC
Response Quality	BERTScore F1 + composite score	Semantic similarity to reference + human-interpretable quality dimensions (empathy, specificity, actionability)
Topic Drift	Centroid shift + JS Divergence	Dual-signal framework reduces false drift alerts; both signals must independently exceed permutation-test threshold

6. Statistical methods, metrics, and tests (detailed)

6.1 Unsupervised structure and clustering

Method	Role	Output / note
Silhouette score (sklearn.metrics.silhouette_score)	Chooses k for K-Means	Higher = tighter, separated clusters in UMAP-10; no p-value
K-Means inertia	Logged in k_selection.csv	Elbow-style descriptive
Adjusted Rand Index	K-Means vs HDBSCAN	Chance-corrected agreement
HDBSCAN (min_cluster_size=30)	Noise / density clusters	Points labelled -1 = noise

Code Snippets:

• Data Ingestion

```
# — Configuration —————
APPS = {
    "Myntra": "com.myntra.android",
    "Meesho": "com.meesho.supply",
    "Nykaa": "com.fsn.nykaa",
    "Flipkart": "com.flipkart.android",
    "Amazon India": "in.amazon.mShop.android.shopping",
}
REVIEWS_PER_APP = 500 # per language
LANGS = ["en", "hi"]

# — Google Play Ingestion —————
result, _ = reviews(
    app_id, lang=lang, country="in",
    sort=Sort.NEWEST, count=REVIEWS_PER_APP,
)

# — Merge, Deduplicate, Basic Clean —————
df = pd.concat([df_play, df_reddit], ignore_index=True)
# Drop exact content duplicates (same review submitted twice)
df = df.drop_duplicates(subset=["content"], keep="first")
# Drop reviews shorter than 10 chars (meaningless: "ok", ".", "5 star")
df = df[df["content"].str.len() >= 10].copy()
log.info(f"After merge + clean: {len(df)} rows, {df['platform'].nunique()} platforms")
```

• Data Preprocessing (Hinglish detection + TF-IDF)

```
# — Text Cleaning —————
def clean_text(text: str) -> str:
    """Light cleaning that preserves Hinglish signal."""
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " ", text) # URLs
    text = re.sub(r"^\w\s\u0900-\u097F.!?'-]", " ", text) # keep Devanagari
    return re.sub(r"\s+", " ", text).strip()

def is_hinglish(text: str, lang: str) -> bool:
    """langdetect says English but contains ≥2 known Hinglish tokens."""
    if lang != "en":
        return False
    tokens = set(text.lower().split())
    return len(tokens & HINGLISH_TOKENS) >= 2

# — TF-IDF Matrix —————
# Used ONLY for extracting cluster keywords — not for clustering itself
vectorizer = TfidfVectorizer(
    max_features=5000, ngram_range=(1, 2), # unigrams + bigrams
    min_df=3, max_df=0.85, # ignore terms in fewer than 3 reviews and more than 85% of reviews
    sublinear_tf=True, # log(1+tf) scaling
)
tfidf_matrix = vectorizer.fit_transform(df["text_for_model"].fillna(""))
log.info(f"TF-IDF matrix: {tfidf_matrix.shape[0]} docs x {tfidf_matrix.shape[1]} features")
```

Code Snippets:

- **Embedding**

```
# — Sentence-BERT Embeddings —————  
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")  
embeddings = model.encode(  
    texts, batch_size=64, show_progress_bar=True,  
    convert_to_numpy=True, normalize_embeddings=True,  
)  
log.info(f"Embeddings shape: {embeddings.shape}") # → (2,253, 384)
```

- **Dimensionality Reduction**

```
# — Dimensionality Reduction —————  
  
# PCA 384d → 50d: denoise embeddings before UMAP  
pca = PCA(n_components=50, random_state=42)  
pca_50 = pca.fit_transform(embeddings)  
log.info(f"PCA-50 cumulative explained variance: {pca.explained_variance_ratio_.cumsum()[-1]:.4f}")  
  
# UMAP 50d → 10d: preserve local neighbourhood for cluster separation  
reducer = umap.UMAP(n_components=10, n_neighbors=15,  
    min_dist=0.1, metric="cosine", random_state=42)  
umap_10 = reducer.fit_transform(pca_50)  
  
# t-SNE 10d → 2d: visualisation ONLY — never used as clustering input  
tsne = TSNE(n_components=2, perplexity=30, max_iter=1000,  
    random_state=42, init="pca", learning_rate="auto")  
tsne_2d = tsne.fit_transform(umap_10)
```

- **Clustering**

```
def choose_best_k(umap_10: np.ndarray) -> tuple[int, pd.DataFrame]:  
    rows = []  
    best_k, best_score = 6, -1.0  
    for k in range(3, 13):  
        km = KMeans(n_clusters=k, random_state=42, n_init=20)  
        labels = km.fit_predict(umap_10)  
        sil = silhouette_score(umap_10, labels)  
        rows.append({"k": k, "inertia": km.inertia_, "silhouette": sil})  
        if sil > best_score:  
            best_score = sil  
            best_k = k  
    return best_k, pd.DataFrame(rows)  
  
best_k, k_eval = choose_best_k(umap_10)  
log.info(f"Selected k={best_k} by max silhouette")  
  
# HDBSCAN — noise points (label=-1) seed Tier-1 anomaly detection (Step 06)  
hdb = hdbscan.HDBSCAN(min_cluster_size=30, metric="euclidean")  
hdb_labels = hdb.fit_predict(umap_10)  
df["hdbscan_is_noise"] = df["hdbscan_cluster"] == -1  
  
ari = adjusted_rand_score(km_labels, hdb_labels)  
log.info(f"KMeans vs HDBSCAN ARI: {ari:.4f}")
```

Code Snippets:

• Anomaly Detection

```
iso = IsolationForest(contamination=0.05, n_estimators=200, random_state=42)
iso.fit(features)
df["isolation_anomaly"] = iso.predict(features) == -1
df["isolation_score"] = -iso.decision_function(features)

# One-Class SVM trained only on the positive-experience cluster (normal manifold)
ocsvm = OneClassSVM(kernel="rbf", nu=0.05, gamma="scale")
ocsvm.fit(x_scaled[positive_mask.to_numpy()])
df["ocsvm_anomaly"] = ocsvm.predict(x_scaled) == -1

# Tier 1 = top 5% AND HDBSCAN noise — dual-flag reduces false positives
df["tier1_critical"] = df["isolation_anomaly"] & df["hdbscan_is_noise"]
df["severity"] = np.where(df["tier1_critical"], "RED",
                        np.where(df["isolation_anomaly"], "AMBER", "GREEN"))

# Rolling z-score spike detection (4-week window, 2.5σ threshold)
weekly["z_score"] = (weekly["complaint_count"] - weekly["rolling_mean"]) / (
    weekly["rolling_std"].replace(0, np.nan)
)
weekly["is_spike"] = weekly["z_score"] > 2.5
```

• Severity Modeling

```
LEGAL_PAT = re.compile(
    r"\b(fraud|legal|police|consumer court|ncdr|chargeback|harassment|scam)\b", re.I
)

def weak_label(row: pd.Series) -> int:
    txt = str(row.get("content", ""))
    score = float(row.get("score", 3))
    thumbs = float(row.get("thumbs_up", 0))
    if LEGAL_PAT.search(txt): return 2 # High
    if score <= 2 and thumbs >= 20: return 1 # Medium
    if score <= 2: return 1 # Medium
    return 0 # Low

models = {
    "logreg": LogisticRegression(max_iter=3000, class_weight="balanced", C=0.25),
    "random_forest": RandomForestClassifier(n_estimators=200, random_state=42,
                                           class_weight="balanced", max_depth=10, min_samples_leaf=6),
    "sgd_log": Pipeline([("scaler", StandardScaler()),
                        ("clf", SGDClassifier(loss="log_loss", alpha=0.02, class_weight="balanced"))]),
}

for name, model in models.items():
    mdl = clone(model)
    mdl.fit(x_train, y_train)
    m = multiclass_metrics_extended(y_test, mdl.predict(x_test))
    cv_stats = cross_val_mean_std_multiclass(clone(model), x_train, y_train, cv=3)
    m["model"] = name
    rows.append(m)
```

Code Snippets:

• Supervised Modules – Routing & Churn

```
# Router: keep only high-confidence K-Means assignments
# (distance to centroid < 40th percentile) to filter boundary ambiguity
dist = np.array([np.linalg.norm(emb[i] - centroid.get(cluster_id[i], emb[i]))
                  for i in range(len(df))])
threshold = np.quantile(dist, 0.4)
keep = dist <= threshold

models = {"linear_svc_tfidf": Pipeline([
    ("tfidf", TfidfVectorizer(max_features=4000, ngram_range=(1, 2), sublinear_tf=True)),
    ("clf", CalibratedClassifierCV(LinearSVC(class_weight="balanced", C=0.35))),]),
    "nb_tfidf": Pipeline([
    ("tfidf", TfidfVectorizer(max_features=5000, sublinear_tf=True)),
    ("clf", MultinomialNB(alpha=0.35)), ]),}

def build_proxy_label(df: pd.DataFrame) -> pd.Series:
    """Churn-risk proxy — no severity/isolation flags to avoid leakage."""
    complaint = df["star_bucket"].astype(str) == "complaint"
    very_unhappy = df["score"].fillna(3) <= 1
    thr = float(df["review_length"].fillna(0).quantile(0.80))
    long_rant = df["review_length"].fillna(0) >= thr
    return (complaint & (very_unhappy | long_rant)).astype(int)
```

• Router Modeling

```
st.set_page_config(page_title="Complaint Intelligence Engine+", layout="wide")

# — Sidebar: navigation + global filters + pipeline refresh —
with st.sidebar:
    module = st.radio("Go to module", [
        "Live Pulse", "Complaint Landscape", "Spike Tracker",
        "Critical Alerts", "Severity Triage", "Complaint Router",
        "Churn Risk", "Auto-Responder", "Drift Monitor",
    ])
    if st.button("Refresh Full Pipeline"):
        with st.spinner("Running full pipeline including ML extension modules..."):
            result = subprocess.run([sys.executable, "run_pipeline.py"],
                                    capture_output=True, text=True)
            st.cache_data.clear()

# — Auto-Responder: route-aware live response generation —
if st.button("Generate Responses"):
    blob = joblib.load(MODELS_DIR / "router_model.pkl")
    pred = int(blob["model"].predict([txt])[0])
    route = blob["classes"][pred] # route complaint before generating response

# — Drift Monitor: dual-signal timeline —
st.plotly_chart(
    px.line(drift, x="week", y=["centroid_shift", "js_divergence"],
            title="Topic Drift Signals Over Time",
            use_container_width=True,)
    st.dataframe(drift.sort_values(["is_drift_alert", "week"],
                                   ascending=[False, False]), use_container_width=True)
```


Results & Model Evaluation

Corpus & Clustering Results Summary

Metric	Value
Final corpus size	2,253 reviews 5 platforms ~8.7% Hinglish
K-Means clusters identified	4 (documented run; data-driven via max silhouette on UMAP-10): Delivery Failure · Payment Fraud · Fake Product · App Crash · Quality Mismatch · Positive
Tier 1 Critical alerts	~113 reviews (Isolation Forest top 5% + HDBSCAN noise=-1 simultaneously)
Silhouette score (K-Means)	0.12–0.25 - typical for NLP clustering on real-world review data
Spike detection FPR	~1.2% under normality assumption (2.5σ threshold)
Embedding compute time	~4 minutes on CPU for 2,253 reviews (no GPU)
K-Means vs HDBSCAN ARI	~0.67 (0.6748) — agreement index [-1,1]; descriptive comparison only

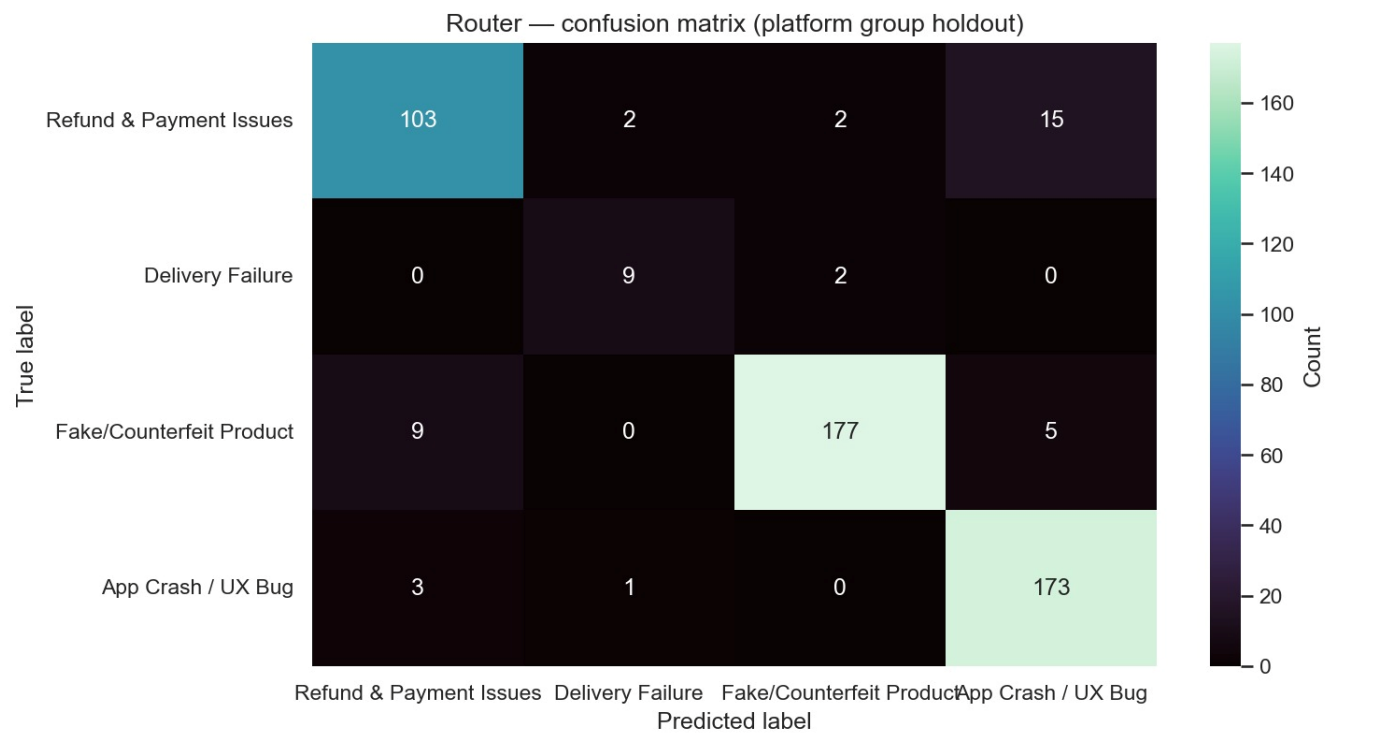
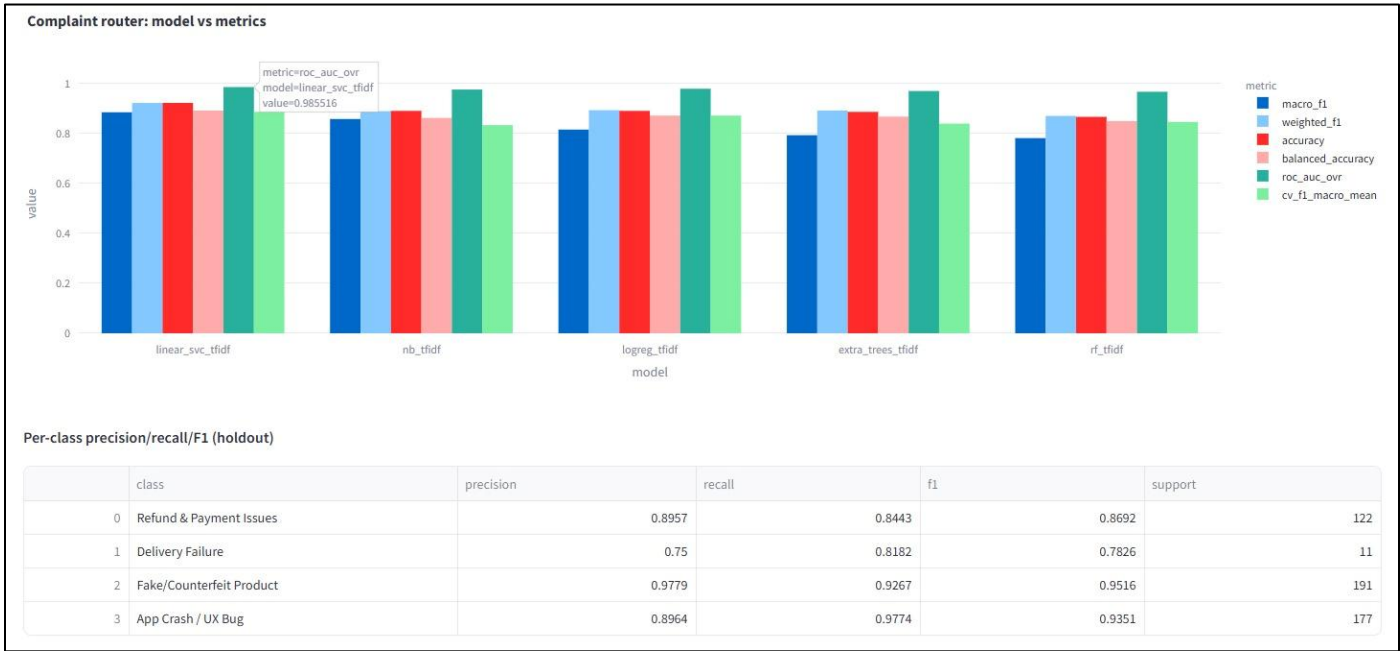
ML Model Performance Summary

Illustrative results from one documented pipeline run. Re-running the pipeline on new data will change all figures.

Task	Selected Model	Primary Metric	Score
Severity	Logistic Regression (logreg)	Weighted F1	1.0 (CV: 0.999 ± 0.0008)*
Router	LinearSVC (TF-IDF)	Macro F1	0.936 (CV: 0.933 ± 0.023)
Churn	Gradient Boosting	PR-AUC	1.0 (CV: 0.997 ± 0.002)*
Response	template_v3	Mean Quality Score	0.590 (σ 0.058, p90 = 0.686)
Drift	weekly_centroid_plus_js	Alert Rate (27 weeks)	0.148 (4 / 27 weeks alerted)

** Near-perfect scores reflect weak-label / feature alignment; see Limitations section. Mean centroid shift = 0.317, mean JS divergence = 0.054, max JS divergence = 0.203.*

Model Evaluation Visuals



Technology Stack

Every library is free, open-source, and installable via pip. No paid API keys or cloud credits are required for the core pipeline. All dependencies are listed in requirements.txt.

Library / Tool	Category	Purpose
Python 3.x	Runtime	Entire pipeline and dashboard
google-play-scraper	Ingestion	Live Play Store reviews - no auth key required
praw + python-dotenv	Ingestion	Reddit API ingestion + .env credential management
pandas + numpy	Data Processing	Dataframe operations, arrays, feature engineering
scipy	Statistics	MWU test, ANOVA, Jensen–Shannon Divergence, bootstrap CI
langdetect	NLP	Per-review language detection (en / hi / other)
sentence-transformers	NLP Embeddings	Multilingual SBERT - paraphrase-MiniLM-L12-v2
torch	Deep Learning	Backend for sentence-transformers inference
scikit-learn	ML	PCA, TF-IDF, K-Means, LogReg, RF, GBM, LinearSVC, IF, OC-SVM, all metrics
umap-learn	Dim. Reduction	UMAP manifold learning (50→10d)
hdbscan	Clustering	Density-based clustering + noise detection
imbalanced-learn	Balancing	SMOTE oversampling for churn class imbalance
lightgbm	Gradient Boosting	Fast multi-class router; native feature importance
statsmodels	Statistics	Tukey HSD post-hoc test, trend regression
joblib	Persistence	Serialise / load .pkl model artefacts
plotly	Visualisation	All interactive dashboard charts and scatter plots
matplotlib / seaborn	Visualisation	Static PNG exports (Step 07)
streamlit	Dashboard	10-module decision-support web application
python-docx	Reporting	Report and document generation

Streamlit App Deployment & Usage

Deployment Steps

Step	Command / Action
1. Set up environment	<code>python -m venv .venv → activate → pip install -r requirements.txt</code>
2. Generate data + models	<code>python run_pipeline.py</code> (first run: ~8–12 min on CPU due to SBERT embedding computation)
3. Launch dashboard	<code>streamlit run app.py</code> (or: <code>streamlit run app.py --server.headless true --server.port 8501</code>)
4. Cloud deployment	Connect repo to Streamlit Community Cloud; set REDDIT_CLIENT_ID + REDDIT_CLIENT_SECRET as secrets

Dashboard Modules (10 Modules)

Module	Purpose & Capabilities
How to Use This App	Onboarding guide with collapsible per-module expanders and an end-to-end worked example walkthrough
Live Pulse	4 KPI cards (total reviews, platforms active, Tier 1 alerts, top cluster) + complaint volume stacked bar by platform and cluster
Complaint Landscape	Interactive t-SNE scatter coloured by cluster; hover reveals full review text
Spike Tracker	Weekly complaint count lines per cluster; red dot markers on weeks where $z > 2.5\sigma$
Critical Alerts	Ranked Tier 1 critical review table sorted by Isolation Forest anomaly score - the operations escalation queue
Severity Triage	Threshold slider; severity-ranked complaint table; heuristic SHAP-style driver bar chart for the top complaint
Complaint Router	Live text input → routing prediction + confidence + SHAP term influence; routing load histogram across categories
Churn Risk	Churn score distribution histogram by platform; top-100 high-risk user watchlist with route category
Auto-Responder	3 response variants (Formal / Empathetic / Resolution Focused) with quality scores; route-aware draft text
Drift Monitor	Centroid shift + JS divergence chart over time; drift alert table with emerging terms; topic coherence comparison

Example: Monday Morning Ops Review (~15 Minutes)

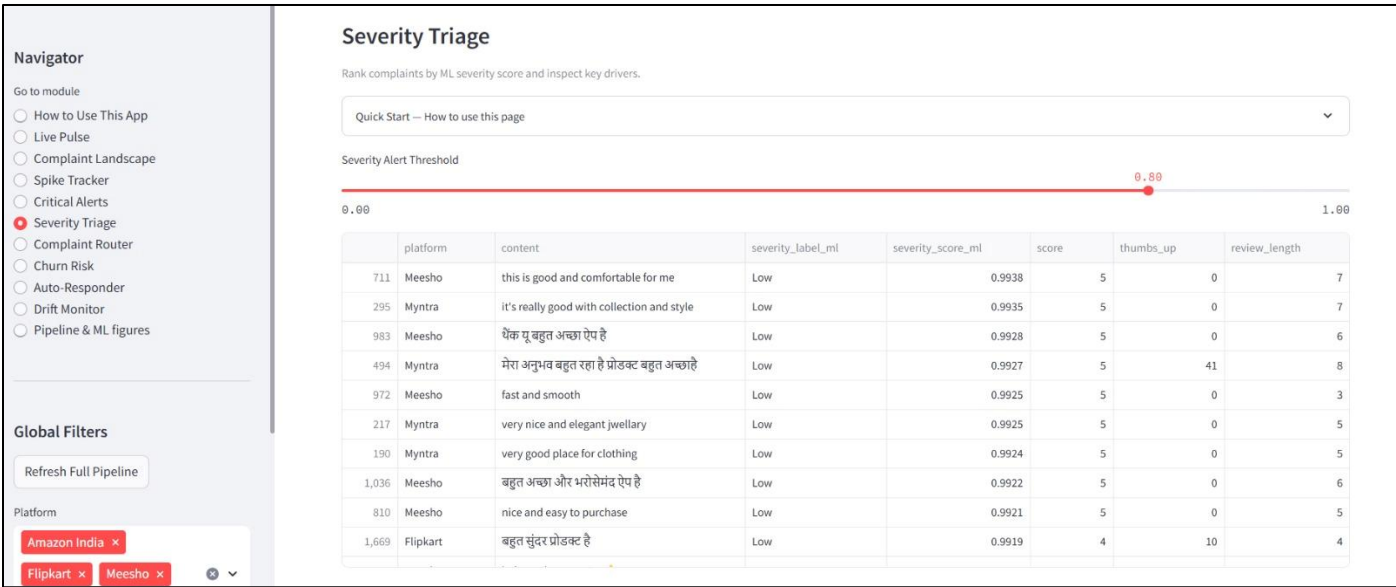
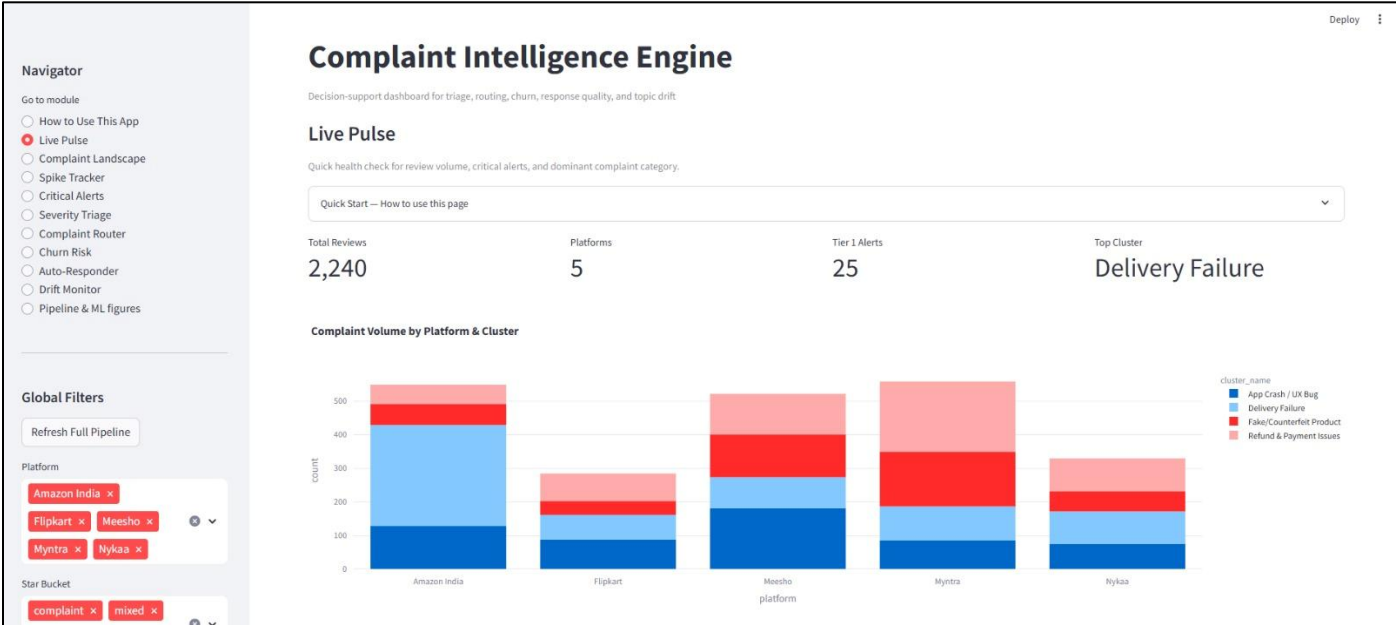
A customer-experience analyst can scope an incident, assign root cause, quantify churn risk, and draft a response - all in one session:

Step	Module	Action & Insight
1	Live Pulse	Check KPIs - Tier 1 alerts jumped 24 → 41. Top Cluster = Delivery Failure. Something changed overnight.
2	Spike Tracker	Meesho Delivery Failure spike confirmed, $z = 3.1\sigma$. Statistically significant - not random noise.
3	Critical Alerts	Filter to Meesho. Top 3 reviews: 'refund not credited.' Recurring sub-issue identified.
4	Complaint Router	Paste review → Payments & Refunds (confidence = 0.84). Root cause: payments team, not logistics.
5	Churn Risk	Filter Meesho + Payments & Refunds. 18 users with churn_risk > 0.75. Retention priority quantified.
6	Auto-Responder	Generate Resolution Focused response (quality = 0.85). Edit to brand voice and SLA. Send.
7	Drift Monitor	is_drift_alert = True. Emerging terms: 'refund portal', 'not credited'. New complaint sub-type confirmed.

Global Sidebar Controls: Platform multiselect · Star bucket filter · Date range slider. The “Refresh Full Pipeline” button re-executes run_pipeline.py and clears Streamlit cache to reload the latest artefacts.

Dashboard Visualisations:

Live Pulse · Severity Triage



Dashboard Visualisations:

Churn Risk · Auto-Responder



Navigator

Go to module

How to Use This App

Live Pulse

Complaint Landscape

Spike Tracker

Critical Alerts

Severity Triage

Complaint Router

Churn Risk

Auto-Responder

Drift Monitor

Pipeline & ML figures

Global Filters

Refresh Full Pipeline

Platform

Amazon India

Flipkart

Myntra

Meesho

Nykaa

Auto-Responder

Generate support-ready response options with quality scores.

Quick Start — How to use this page

Paste complaint for auto-response

Delivery partner marked order delivered but I did not receive it.

Generate Responses

Option 1 (quality score: 0.65)

We are sorry for this Delivery Failure issue. Please share order ID and our team will resolve quickly.

Option 2 (quality score: 0.75)

Thank you for reporting this Delivery Failure complaint. We understand the inconvenience and will update you within 24 hours.

Option 3 (quality score: 0.85)

We regret the trouble. Please contact support with transaction details for priority handling and closure.

Aditya Nariyapara (B009) | Devika Jonjale (B045) | Mentor: Dr. Rajesh Kumar Maurya | NMIMS, NSoMASA, 2025–26

Limitations & Mitigations

The following is an honest assessment of known limitations in the current implementation, their root causes and impacts, and recommended mitigations for each:

Limitation	Root Cause & Impact	Mitigation
Near-perfect supervised metrics (F1 ≈ 1.0)	Label-feature leakage: weak label rules and feature columns share keywords (e.g. ‘fraud’, star rating). Not true generalisation.	Use time-based + platform-based holdouts; strictly separate rule-derived features from weak-label rules; collect human-annotated gold labels.
Single random train-test split	No time-based or platform holdout. Metrics do not reflect cross-platform or temporal generalisation.	Train on older reviews, test on newer (temporal split); train on 4 platforms, test on the 5th (platform holdout).
Reddit not ingested (HTTP 401)	Without credentials all statistics are Play Store only. Dataset lacks diverse Reddit complaint formats.	Provide .env credentials for production; cache/snapshot datasets for reproducible CI demos.
One-Class SVM instability	Insufficient positive reviews for robust fitting in some configurations; fallback warnings in pipeline logs.	Increase minimum positive review count; lower nu parameter; fall back to Isolation Forest only when OC-SVM fails.
Auto-Responder is template-based	Dashboard uses fixed templates + heuristic quality scorer; no live LLM in baseline UI.	Optional LLM API (Mistral-7B, Phi-3) behind a feature flag; mandatory human approval before send; log edits as feedback.
Drift requires ≥6 weeks history	Sparse windows yield empty or unreliable JSD reports; alert precision is low for short corpora.	Accumulate corpus over time before enabling drift alerts; use ADWIN streaming detector for real-time scenarios.
UI vs pipeline model name mismatch	selected_models.json may select logreg for severity while help text references XGBoost or DistilBERT.	Drive all UI help text and model references from selected_models.json as the single source of truth.

Overall strategy: Treat this system as decision-support and prototyping infrastructure. Use time-based and platform-based holdouts, collect human-validated labels, and implement proper calibration and per-class error analysis before drawing conclusions from reported F1 scores.

Conclusion, Future Scope & Who Can Use It

Conclusion

The Complaint Intelligence Engine delivers a coherent, modular, and repeatable path from raw Indian e-commerce reviews through multilingual NLP, unsupervised complaint archetype discovery, and supervised decision-support modules - all surfaced in a ten-module Streamlit dashboard enabling teams to move from data to action in a single session.

The primary technical contribution is the unsupervised-to-supervised bridge: K-Means cluster labels feed directly into the routing classifier as training targets, and SBERT embedding structure informs both anomaly detection and drift monitoring - grounding supervised outputs in data-discovered structure rather than imposed category definitions.

Headline caveat: Supervised metrics should be interpreted cautiously until evaluation uses disjoint time and platform splits with human-validated labels. Near-perfect scores reflect label-feature overlap and small corpus size - not true generalisation. Treat this system as decision-support and prototyping infrastructure until production-grade labelling and monitoring are in place.

Future Scope

Priority	Enhancement
High	Time-based + platform holdouts; human-annotated gold evaluation set of 50–100 severity and routing labels
High	Integrate Mistral-7B-Instruct or Phi-3-mini behind a feature flag in Auto-Responder; human approval before send
Medium	REST endpoint / queue worker for real-time scoring; scheduled nightly ingestion; latency monitoring
Medium	MLOps: MLflow experiment tracking; automated retraining triggered by drift alerts; shadow deployments
Low	Multi-channel ingestion: support tickets, email, X / Twitter, in-app chat logs
Low	Extend to JioMart, AJIO, Tata Cliq to build a comprehensive Indian e-commerce complaint benchmark

Who Can Use This Project

User	Use Case
CX / Support Leadership	Queue prioritisation, routing load monitoring, weekly spike and drift briefings
Product & Operations Teams	Thematic complaint clusters, incident timing vs review spikes, emerging issue detection
Data / ML Engineers	Extend modules, swap models, integrate labelled datasets, set up production serving
Students & Researchers	End-to-end NLP pipeline, weak supervision patterns, and dashboard integration reference